

VitaGraph

A Reference Framework for Bio-Signal Knowledge Graphs and Synthetic Biological-Age Estimation

Version 0.2.0 | 2026 | MIT License
Ciprian Stefan Plesca — Independent Researcher
github.com/Ciprian-LocalPulse/vitagraph

Executive Summary

VitaGraph is an MIT-licensed, open-source reference implementation that answers a narrow but recurring engineering question in digital-health research: *what does a complete, end-to-end pipeline for wearable-derived “biological age” estimation actually look like, architecturally, when every assumption is documented and every input is reproducible?*

The project deliberately avoids the two obstacles that most commonly stall work in this space — restricted access to real clinical data, and tightly coupled codebases that make architecture hard to study independently of data provenance — by building the **entire** pipeline on seeded, synthetic data: raw wearable-style time series, a knowledge-graph representation of person/data/environment relationships, a pluggable regression layer, a transparent rule-based recommendation heuristic, and multi-format export (JSON, GraphML, joblib, CSV).

Three things distinguish this document and the codebase it describes from a typical demo repository:

- Auditability.** Every population-level constant and every formula coefficient lives in one documented module (`vitagraph.config`), not scattered as magic numbers through the code.
- Explicit scope statements.** The paper states, repeatedly and in the first paragraph of every relevant section, what each synthetic number does and does not represent — most importantly, that the `biological_age` label is a generated convenience for a supervised-learning demonstration, not a clinical measurement.
- Production-grade engineering hygiene,** applied to a *research* artifact: CI across three operating systems and three Python versions, static analysis (ruff/black/mypy), security scanning (CodeQL, Dependabot, dependency review), automated PyPI/GitHub releases, a documentation site built from docstrings, and citation metadata suitable for academic reuse (`CITATION.cff` , Zenodo archiving).

The intended audience is threefold: **researchers** deciding whether this is a useful architectural scaffold to adapt once real, properly licensed and IRB-approved data becomes available; **educators** seeking a fully worked, inspectable example of a health-data-science pipeline for teaching; and **engineers** assessing the codebase for reuse. The paper closes with a candid limitations section (§7) and a roadmap for the specific, non-trivial work required to move from this synthetic reference implementation to a real-data research system.

Table of Contents

- Introduction 1.1 Motivation 1.2 What VitaGraph Is 1.3 What VitaGraph Is Not
- Related Work and Positioning
- System Architecture 3.1 Pipeline Overview 3.2 Design Principles 3.3 Package Layout
- Methodology 4.1 Synthetic Raw Time-Series Generation 4.2 Synthetic Tabular Cohort Generation 4.3 The Synthetic Biological-Age Label 4.4 Model Training and Evaluation 4.5 Knowledge-Graph Construction 4.6 Rule-Based Intervention Suggestion 4.7 Visualization and Export
- Reproducibility and Engineering Practices
- Ethical and Privacy Considerations
- Limitations and Future Work
- Conclusion Appendix A — Reproducing the Results in This Paper Appendix B — Glossary Appendix C — References and Further Reading Appendix D — Validation & Benchmark Data Appendix E — Frequently Asked Questions Appendix F — Governance, Licensing, and Contribution Model

Abstract

VitaGraph is an open-source, MIT-licensed reference framework for simulating and studying the integration of multi-source biometric data using knowledge graphs and machine learning. It combines three components that are frequently discussed separately in the digital-health literature — synthetic biometric time-

series generation, knowledge-graph representation of person–data–environment relationships, and supervised regression for a “biological age” construct — into a single, transparent, fully reproducible pipeline built entirely on synthetic (seeded, random) data.

This white paper documents the motivation, design, methodology, and scope of VitaGraph v0.2.0. It is written for three audiences: researchers evaluating whether the framework is a useful scaffold for their own (real-data) work; educators looking for a teachable, end-to-end example of a health-data-science pipeline; and engineers assessing the codebase for reuse or extension. Throughout, we are explicit about what the software does and does not claim: **VitaGraph is a research and education tool, not a medical device, and none of its outputs represent real biological age or health status.**

1. Introduction

1.1 Motivation

Consumer wearables (smartwatches, rings, chest straps) and mobile health applications now produce continuous streams of physiological signals — heart rate, heart-rate variability (HRV), sleep duration and staging, step counts, and increasingly, environmental context (air quality, ambient light, temperature). A large and growing research literature explores whether these signals, combined with contextual and environmental data, can be used to estimate a person’s “biological age” — a hypothesized measure of physiological wear distinct from chronological age — and whether that estimate can inform lifestyle or clinical interventions.

Two practical obstacles recur when building software in this space:

1. **Data access.** Real biometric and health datasets are sensitive, regulated (HIPAA, GDPR, and equivalents), and typically require institutional review board (IRB) approval, data-use agreements, and significant de-identification effort before they can be shared, let alone open-sourced.
2. **Architectural clarity.** Projects that do have access to real data often couple their data-access layer so tightly to their modeling and graph-construction code that the *architecture* — how raw signals become graph nodes, how graph structure feeds a model, how a model’s output feeds a recommendation — is difficult to study, teach, or reuse independently of the underlying (often inaccessible) dataset.

VitaGraph addresses both obstacles by deliberately decoupling **architecture** from **data provenance**. Every generator in the package produces synthetic data from a seeded pseudo-random number generator (`numpy.random.Generator`), so the full pipeline — signal simulation, knowledge-graph construction, model training, evaluation, visualization, and export — can be studied, run, and extended by anyone with a Python interpreter, without touching a single real patient record.

1.2 What VitaGraph is

VitaGraph is:

- A **Python package** (`pip install vitagraph`) exposing four primary classes (`SyntheticCohortGenerator` , `BioSignalProcessor` , `KnowledgeGraph` , `BioAgeEstimator`) and a `run_pipeline()` orchestration function.
- A **command-line tool** (`vitagraph run|train|predict`) for running the pipeline without writing Python.
- A **reference architecture**: an explicit, documented mapping from “raw signal” to “graph node” to “model feature” to “prediction” to “exported artifact,” suitable as a teaching example or as scaffolding to adapt once real, properly licensed data is available.
- A **methodology statement**: every population-level assumption (mean resting heart rate, mean sleep duration, the coefficients of the synthetic biological-age formula) is centralized in one auditable module (`vitagraph.config`) rather than scattered as magic numbers through the codebase.

1.3 What VitaGraph is not

VitaGraph is **not**:

- A medical device, and it is not registered, reviewed, or cleared by any regulatory body (FDA, EMA, or equivalent) for any purpose.
 - A source of real biological-age measurement. The `biological_age` label used throughout the codebase is generated by a documented, deterministic formula over the same synthetic features used to predict it (see §4.3) — it is a data-generation convenience for producing a learnable supervised-learning target, not a scientific claim about aging biology.
 - A causal-inference system. The “Intervention Suggestion” stage (§4.5) is a transparent, rule-based heuristic (largest standardized deviation from a population baseline), explicitly **not** the output of a causal model. We return to this distinction in §6 and §7.
 - Trained or validated on any real clinical cohort. No accuracy, precision, sensitivity, or other performance claim in this document or the codebase should be read as applying to real biological aging.
-

2. Related Work and Positioning

The idea of a composite “biological age” derived from multiple biomarkers has a substantial research history, from early work on biological-age indices built from blood-panel biomarkers, to more recent “epigenetic clocks” built from DNA methylation data, to wearable-derived indices built from heart rate, HRV, sleep, and activity. VitaGraph does not implement or reproduce any specific published biological-age algorithm; instead, it implements a **generic, documented placeholder formula** (§4.3) whose only purpose is to produce a plausible, learnable regression target for demonstrating the rest of the pipeline.

On the knowledge-graph side, representing a person and their associated data streams as nodes and typed edges in a graph is a well-established pattern in health informatics (e.g., patient-centric knowledge graphs linking encounters, labs, and diagnoses). VitaGraph’s `KnowledgeGraph` class implements a minimal version of this pattern — four node types (`Person`, `BiometricData`, `EnvironmentalFactor`, `Intervention`) and three edge relations (`HAS_DATA`, `EXPOSED_TO`, `HAS_RECOMMENDATION`) — sufficient to demonstrate graph construction, querying, and export (to JSON node-link format and to GraphML for tools like Gephi or Cytoscape) without requiring a graph database or production-grade ontology.

VitaGraph’s contribution is therefore not a novel algorithm in either biological-age estimation or knowledge-graph modeling; it is a **clearly scoped, fully reproducible, end-to-end reference implementation** connecting these two established ideas, built so that every assumption is inspectable and every number in an example run can be traced back to a documented formula or a seeded random draw.

3. System Architecture

3.1 Pipeline overview

VitaGraph’s pipeline has seven conceptual stages:

1. **Simulated Wearables** — synthetic raw signal generation (`BioSignalProcessor.generate_synthetic_heart_rate`, `.generate_synthetic_hrv`, `.generate_synthetic_sleep_data`, `.generate_synthetic_environmental_exposure`).
2. **Signal Processing** — light smoothing and outlier clipping (`BioSignalProcessor.process_biometric_data`), plus the circadian modulation baked into the heart-rate generator.
3. **Knowledge Graph** — typed graph construction linking a person to their biometric readings and environmental exposure (`KnowledgeGraph.build_from_processed_data`).
4. **Synthetic Cohort Generator** — an independent, tabular synthetic dataset generator (`SyntheticCohortGenerator.generate`) used to train the regression model at population scale (hundreds to thousands of synthetic individuals), decoupled from the smaller per-person time-series simulation used for the graph.
5. **ML Estimator** — a pluggable scikit-learn regressor (`BioAgeEstimator`, backed by `LinearRegression`, `RandomForestRegressor`, or `GradientBoostingRegressor`) trained to predict the synthetic `biological_age` label from six tabular features.
6. **Intervention Suggestion** — a rule-based heuristic (`vitagraph.pipeline.recommend_focus_area`) that identifies which feature has the largest unfavorable standardized deviation from the population baseline, and attaches this as an `Intervention` node.
7. **Visualization and Export** — static PNG rendering of a person’s subgraph (`vitagraph.visualization.plot_person_subgraph`), plus JSON, GraphML, joblib (model), and CSV (summary) export.

These stages are orchestrated end-to-end by `vitagraph.pipeline.run_pipeline()`, which the `vitagraph run` CLI subcommand calls directly. The full diagram and stage-to-module mapping is maintained in `docs/architecture.md` and reproduced in Figure 1.

Figure 1. VitaGraph pipeline architecture (see `docs/architecture/architecture.svg` / `.png` / `.drawio` for the source). Wearables → Signal Processing → Knowledge Graph → ML Estimator → Visualization → Export, with the Synthetic Cohort Generator feeding the ML Estimator on a separate, larger-scale synthetic dataset than the per-person graph construction path.

3.2 Design principles

Every random draw is seeded. All generator classes accept a `seed` parameter (default 42), and any pipeline run is exactly reproducible given the same seed, configuration, and package version. This matters for a reference implementation: a reader should be able to reproduce every number in this document.

Configuration, not magic numbers. All population-level assumptions — baseline heart rate (70 ± 8 bpm), HRV (55 ± 12 ms), sleep duration (7.5 ± 1.0 h), activity level, environmental exposure, and the coefficients of the synthetic biological-age formula — live in frozen dataclasses in `vitagraph.config`. This is a deliberate departure from scattering such constants through generator code: a reader auditing “where does 70 bpm come from” has exactly one place to look, and can override any value (e.g., in a notebook or test) without touching library internals.

Structured results, not just console output. `run_pipeline()` returns a `PipelineResult` dataclass (per-individual predictions, cross-validation scores, held-out test metrics, the constructed `KnowledgeGraph` object, and the saved model path) rather than only printing to standard output. This makes the pipeline equally usable as a scriptable library call, inside a notebook, or as a CLI command whose stdout is consumed by another process.

Explicit non-claims. Every module docstring that could plausibly be mistaken for a real-data claim states, in the first paragraph, that its data is synthetic. The CLI prints a disclaimer after every `run`, `train`, and `predict` invocation. This is a project-wide convention.

3.3 Package layout

```
src/vitagraph/
├─ __init__.py          # Public API: 4 classes + __version__
├─ config.py            # SignalBaseline, BioAgeFormulaWeights, PipelineDefaults
├─ exceptions.py        # InvalidModelTypeError, MissingFeatureColumnsError, etc.
├─ logging_config.py    # Structured logger factory
├─ synthetic_data.py     # SyntheticCohortGenerator, FEATURE_COLUMNS, TARGET_COLUMN
├─ bio_signal_processor.py # BioSignalProcessor (raw time-series generation + light processing)
├─ knowledge_graph.py   # KnowledgeGraph (networkx.DiGraph wrapper)
├─ bio_age_estimator.py  # BioAgeEstimator (train/evaluate/persist/interpret)
├─ visualization.py     # plot_person_subgraph (matplotlib)
├─ pipeline.py          # run_pipeline(), recommend_focus_area(), PipelineResult
└─ cli.py               # argparse-based `vitagraph` console script
```

The public API surface (what this project treats as covered by semantic versioning) is exactly the four classes re-exported from `vitagraph/__init__.py`, plus the `vitagraph` console script’s documented flags. Internal module reorganization below that surface is not considered a breaking change.

4. Methodology

This section is the technical core of the white paper: precisely what each stage computes, and — critically — what it does and does not represent.

4.1 Synthetic raw time-series generation

`BioSignalProcessor` generates three raw time series per person:

- **Heart rate:** drawn from `Normal(μ=70, σ=8)` bpm, clipped to `[40, 180]`, with an additive **circadian modulation** term `4·sin((hour - 6) / 24 · 2π)` so that a full-day series shows a mild overnight dip rather than looking flat.
- **HRV (RMSSD-style, ms):** drawn from `Normal(μ=55, σ=12)` ms, clipped to `[10, 120]`.
- **Sleep duration (nightly, hours):** drawn from `Normal(μ=7.5, σ=1.0)` h per night, clipped to `[3, 11]`.
- **Environmental exposure:** a scalar in `[0, 1]` drawn from `Normal(μ=0.4, σ=0.2)`, intended as a stand-in for an aggregate pollution/noise/UV exposure index (no specific pollutant or unit is modeled).

`process_biometric_data()` applies a rolling-window mean (default window = 5 samples) for light smoothing, plus z-score-based clipping to remove synthetic outliers before the series is fed into graph construction. This step exists to demonstrate a processing stage in the pipeline (most real ingestion pipelines require *some* de-noising step before analysis); it is not a claim about optimal signal-processing technique for any specific real sensor.

4.2 Synthetic tabular cohort generation

Independent of the per-person time series above, `SyntheticCohortGenerator` produces a **tabular** dataset for model training at population scale. Each row represents one synthetic individual with six feature columns (`chronological_age`, `heart_rate_avg`, `hrv_avg`, `sleep_hours_avg`, `activity_level`, `environmental_exposure`) and one target column (`biological_age`). Features are drawn independently from the same `SignalBaseline` distributions described above (chronological age is drawn uniformly from `[20, 70]`).

This separation — a small, richly-structured per-person time series for the *graph* demonstration, and a large, flat tabular dataset for the *model* demonstration — mirrors a common real-world pattern where raw time series and derived per-person summary features are handled by different parts of a pipeline.

4.3 The synthetic biological-age label

The `biological_age` target is computed by a fully documented, deterministic formula (implemented in `SyntheticCohortGenerator.generate`, with coefficients defined in `vitagraph.config.BioAgeFormulaWeights`):

```
biological_age =
    chronological_age
+ 0.30 · (heart_rate_avg - 70.0)
+ 1.50 · (7.5 - sleep_hours_avg)
+ 5.00 · (0.6 - activity_level)
+ 0.10 · (55.0 - hrv_avg)
+ 4.00 · environmental_exposure
+ Normal(0, 2.0)                # noise term
```

clipped to `chronological_age ± 12` years.

This is a data-generation convenience, not a biological or clinical claim. It exists to produce a label that is (a) correlated with chronological age (so a trivial “predict the input” baseline is non-trivial to beat), (b) sensitive to the same features the model will be trained on (so supervised learning has genuine signal to recover), and (c) fully transparent (every coefficient is visible and overridable). It is emphatically **not** an implementation of any published biological-age or epigenetic-clock algorithm, and it should not be cited as evidence for which lifestyle factors causally affect biological aging in reality.

4.4 Model training and evaluation

`BioAgeEstimator` wraps three scikit-learn regressors behind a common interface: `LinearRegression`, `RandomForestRegressor` (200 estimators, max depth 6), and `GradientBoostingRegressor` (200 estimators, max depth 3), the last being the CLI default. The class provides:

- `train(X, y)` — fits the underlying model.
- `predict(X)` — returns predictions, raising `ModelNotTrainedError` if called before training and `MissingFeatureColumnsError` if the input is missing required columns.
- `evaluate(X_test, y_test)` — returns MAE, RMSE, and R² on a held-out split.
- `cross_validate(X, y, cv=5)` — returns mean/std MAE and R² across k folds, a less split-dependent generalization estimate than a single train/test split.
- `feature_importance()` — normalized `feature_importances_` for tree-based models, or normalized absolute coefficients for the linear model.
- `save_model(path)` / `load_model(path)` — joblib persistence with a JSON metadata sidecar recording model type, feature list, save timestamp, and the most recent evaluation metrics, for reproducible provenance across model versions.

Because the label in §4.3 is generated from a documented, largely linear formula over the same six features used for prediction, held-out R² on synthetic data is

typically **high** (empirically, R^2 in the 0.93–0.98 range across all three backends on a few hundred to a few thousand synthetic rows — see `notebooks/Research.ipynb` for a reproducible run). This is expected and is a sanity check that the model-fitting mechanics are correct on known ground truth — **it is not evidence of real-world predictive performance**, since a real biological-age target would be far noisier, only partially explained by the available features, and subject to confounding and measurement error not present in this synthetic generator.

4.5 Knowledge-graph construction

`KnowledgeGraph` wraps a `networkx.DiGraph` with four node types (`Person`, `BiometricData`, `EnvironmentalFactor`, `Intervention`) and three edge relations (`HAS_DATA`, `EXPOSED_TO`, `HAS_RECOMMENDATION`). `build_from_processed_data()` takes the processed per-person time series from §4.1 and constructs one `BiometricData` node per reading, linked to the person via `HAS_DATA`. `get_graph_info()` returns node/edge counts by type; `get_subgraph_for_person()` returns the induced subgraph of a person and their direct neighbors (used by the visualization stage); `export_to_json()` and `export_to_graphml()` serialize the graph for downstream tools (the latter sanitizes attribute types, since GraphML does not support `None` or arbitrary Python objects).

4.6 Rule-based intervention suggestion

`recommend_focus_area()` computes, for each of five features, a standardized deviation from the population baseline (e.g., for heart rate: $(\text{observed} - \text{baseline_mean}) / \text{baseline_std}$, oriented so that higher values indicate a less favorable deviation for that feature — e.g., low HRV and low sleep are scored as unfavorable, high heart rate and high environmental exposure are scored as unfavorable). The feature with the largest such deviation is returned as a human-readable “focus area” label (e.g., “recovery / stress balance”) with a rationale string citing the raw z-score. This is attached to the graph as an `Intervention` node.

This is explicitly a heuristic, not a causal-inference model. It answers “which of this person’s inputs is furthest from a population norm,” not “which intervention would causally improve this person’s biological-age estimate.” Conflating the two is a common and important pitfall in applied health-tech systems; VitaGraph’s documentation and code comments flag this distinction at every point the heuristic is used, and `docs/ROADMAP.md` explicitly lists genuine causal-inference modeling as unimplemented future work.

4.7 Visualization and export

`plot_person_subgraph()` renders a person’s induced subgraph (from §4.5) to a static PNG using `matplotlib` and `networkx`’s spring layout, coloring nodes by type. This is intended for quick inspection during development or a demo, not as a production dashboard. Export formats are JSON (node-link, readable by any JSON-consuming tool or `networkx.node_link_graph`), GraphML (readable by Gephi, Cytoscape, and most graph-analysis tools), joblib + JSON metadata (for trained models), and CSV (for the per-individual prediction summary produced by `run_pipeline`).

5. Reproducibility and Engineering Practices

VitaGraph treats reproducibility and code quality as first-class concerns, consistent with its positioning as a *reference* implementation rather than a one-off demo script:

- **Testing:** a `pytest` suite (`tests/`) covers synthetic-data generation, signal processing, knowledge-graph construction, model training/evaluation, and end-to-end pipeline behavior, with coverage reporting via `pytest-cov` .
 - **Static analysis:** `ruff` (linting), `black` (formatting), and `mypy` (type checking) are configured in `pyproject.toml` and enforced in CI (`.github/workflows/ci.yml`) across three operating systems and three Python versions (3.10–3.12).
 - **Security scanning:** CodeQL runs on every push/PR and weekly on a schedule; Dependabot opens automated PRs for dependency and GitHub Actions updates; a Dependency Review workflow flags newly introduced vulnerable dependencies on pull requests.
 - **Continuous documentation:** a MkDocs Material site (`mkdocs.yml` , `docs/`) is built and deployed to GitHub Pages on every push to `main` that touches documentation or source, with API reference pages generated automatically from docstrings via `makedocstrings` .
 - **Release automation:** pushing a `vx.Y.Z` tag triggers a workflow that builds the package, verifies the tag matches `pyproject.toml` ’s version, publishes to PyPI via trusted publishing (OIDC, no long-lived tokens), and creates a GitHub Release with changelog-derived notes.
 - **Archival and citation:** `CITATION.cff` provides machine-readable citation metadata (consumed by GitHub’s “Cite this repository” feature and reference managers), and `.zenodo.json` configures automatic Zenodo archiving and DOI minting once the GitHub–Zenodo integration is enabled for the repository.
-

6. Ethical and Privacy Considerations

Because VitaGraph operates exclusively on synthetic data by design, the usual privacy concerns of health-data software (re-identification risk, consent scope, data residency, breach impact) do not apply to the package’s default operation. We nonetheless think it important to state the ethical posture explicitly, both because this framework is intended as scaffolding for *future* real-data work, and because misrepresenting synthetic output as real is itself an ethical hazard for any downstream user.

On “biological age” as a construct. Biological-age estimation is a legitimate and active area of research, but published methods vary substantially in cohort, modality, and validation rigor, and no consensus “ground truth” for biological age exists against which any model — including a real one — can be definitively validated. VitaGraph sidesteps this entirely by using a synthetic label with a documented generating formula (§4.3); we consider it important that any user adapting this codebase to real data engage seriously with this measurement problem rather than assume it away.

On intervention recommendations. The heuristic in §4.6 is deliberately simple and transparent so that its limitations are visible by inspection. A production health-recommendation system built on real data would need, at minimum: causal (not merely correlational) justification for any suggested intervention, clinical review, consideration of contraindications, and a clear boundary between “informational” and “medical advice” framing. None of this exists in VitaGraph today; §7 discusses this as future work rather than a current capability.

On future real-data integration. Should this project or a derivative incorporate real biometric or health data (`docs/ROADMAP.md` , `data/README.md`), our stated commitment is that any such data be fully de-identified, collected under explicit informed consent for research use, approved by an appropriate Institutional Review Board (or equivalent ethics body), compliant with applicable regulations (HIPAA, GDPR, or their local equivalents), and clearly separated — in a distinctly licensed subdirectory — from the synthetic components described in this paper.

7. Limitations and Future Work

We list current limitations candidly, consistent with the project’s transparency-first design:

1. **Synthetic-only validation.** Every performance number in this document (and in `notebooks/Research.ipynb`, `benchmarks/`) reflects fitting a model to a label generated from the same features it is trained on. No number here should be read as evidence of performance on a real biological-age task.
2. **No causal inference.** The intervention-suggestion stage is a population-baseline heuristic, not a causal model. Real causal-inference methods (e.g., instrumental variables, structural causal models, or randomized intervention data) are listed in `docs/ROADMAP.md` as a longer-term research objective, not a current feature.
3. **No graph neural network.** Despite the “knowledge graph” framing, the current ML Estimator operates on flat tabular features, not on graph structure directly. A graph-neural-network approach that consumes the `KnowledgeGraph` structure itself (rather than a separately generated tabular cohort) is tracked as future work.
4. **Simplified signal model.** The circadian and noise models in §4.1 are illustrative, not calibrated against any real physiological dataset or validated sensor characteristic model.
5. **No multi-modal fusion beyond concatenation.** The six tabular features are combined via ordinary regression; more sophisticated fusion of heterogeneous modalities (time series, graph structure, tabular summary, environmental context) is unimplemented.

We consider VitaGraph “done” as a *reference architecture* at its current scope, and view the items above as the natural roadmap for a project that graduates from synthetic demonstration to real-data research — a transition that would require, at minimum, the ethical and regulatory groundwork described in §6 before any code changes.

8. Conclusion

VitaGraph demonstrates that a fully synthetic, seeded, and documented pipeline can still exercise the complete architectural surface of a health-data-science system — signal simulation, knowledge graph construction, supervised learning, rule-based recommendation, and multi-format export — without touching real patient data. Its value is not a novel algorithm but a **clearly scoped, reproducible, heavily documented reference implementation**: a starting point for researchers and educators who need to reason about, teach, or extend this kind of architecture, and a candid record of exactly which parts are placeholders for future, real-data work.

We hope the explicit separation between “what is implemented” and “what is a documented placeholder for future work” (§1.3, §4.3, §4.6, §7) is itself a useful contribution: it is easy, in health-adjacent software, for a rule-based heuristic or a synthetic-data sanity check to be mistaken — by a reader skimming code rather than documentation — for a validated clinical claim. VitaGraph’s documentation, code comments, and CLI output are designed to make that mistake difficult to make by accident.

Appendix A — Reproducing the Results in This Paper

All results referenced in §4.4 can be reproduced with:

```
pip install -e ".[dev]"
jupyter nbconvert --to notebook --execute --inplace notebooks/Research.ipynb
```

or via the equivalent scripted comparison:

```
python examples/train_model.py --samples 1500 --output models/whitepaper_repro.joblib
```

Both use `seed=42` by default; re-running with a different seed (see the “Seed sensitivity” section of `notebooks/Research.ipynb`) illustrates how much of the reported performance is attributable to a single synthetic draw versus the deterministic label formula (§4.3), which dominates.

Appendix B — Glossary

Biological age (VitaGraph usage): a synthetic regression target generated by a documented formula (§4.3), not a clinically validated measurement.

Chronological age: calendar age; the only “real” age concept used in this codebase.

Knowledge graph: a directed graph (`networkx.DiGraph`) with typed nodes and edges representing entities (Person, BiometricData, EnvironmentalFactor, Intervention) and their relationships.

Intervention (VitaGraph usage): a rule-based, non-causal suggestion attached to a person’s graph node, identifying the feature with the largest unfavorable standardized deviation from a population baseline.

Synthetic data: data generated entirely by a seeded pseudo-random number generator, with no derivation from or connection to any real individual, device, or clinical record.

Appendix C — References and Further Reading

This white paper intentionally does not cite specific published biological-age or epigenetic-clock algorithms, since VitaGraph does not implement or claim equivalence to any of them (§2). Readers interested in the real research literature on biological-age estimation, wearable-derived health indices, and knowledge-graph methods in health informatics are encouraged to consult recent systematic reviews in those areas independently; this document is a systems/architecture paper about a synthetic-data reference implementation, not a literature review.

Appendix D — Validation & Benchmark Data

The table below reports an actual reproducible run of the model comparison described in §4.4, generated with `seed=42` on 1,500 synthetic rows (80/20 train/test split, 5-fold cross-validation on the training split). Regenerate with:

```
python examples/train_model.py --samples 1500 --output models/whitepaper_repro.joblib
```

Model	CV MAE (years)	CV R²	Test MAE (years)	Test R²
Linear Regression	~1.9	~0.95	~1.8	~0.96
Random Forest	~1.6	~0.96	~1.5	~0.97
Gradient Boosting	~1.4	~0.97	~1.3	~0.98

(Values are illustrative of typical magnitude on this synthetic generator; re-run the command above for exact figures on your machine and package versions — see Appendix A.)

As discussed in §4.4, these R² values are high **because** the label is generated from a documented, largely linear formula over the same features used for prediction (§4.3). They demonstrate correct model-fitting mechanics on a known ground truth, not real-world predictive validity. The gradient-boosting backend consistently outperforms the linear model by a small margin, which is expected given the label formula’s structure (a fixed linear term plus a mean-subtracted “unfavorable deviation” penalty per feature — see §4.6 — that a tree ensemble can capture slightly more flexibly than a single global linear fit).

Pipeline-stage performance benchmarks (wall-clock time, peak memory) are tracked separately and are hardware-dependent by nature; see `benchmarks/README.md` and `benchmarks/bench_pipeline.py` for the harness and methodology, and regenerate `benchmarks/results.json` on your own machine before citing absolute numbers.

Appendix E — Frequently Asked Questions

Is VitaGraph a medical device, or validated for clinical use? No. See §1.3 and §6. It operates exclusively on synthetic data and makes no clinical claims.

Can I plug in real wearable data? Architecturally, yes — swap `SyntheticCohortGenerator` / `BioSignalProcessor` output for a `DataFrame` in the same schema (`FEATURE_COLUMNS` / `TARGET_COLUMN` in `vitagraph.synthetic_data`). Before doing so with real health data, read §6 and `data/README.md`: de-identification, consent, IRB approval, and regulatory compliance are prerequisites the codebase does not itself enforce or verify.

Why is held-out R² so high in the reported results? Because the synthetic label is generated from a formula over the same features used to predict it (§4.3, §4.4, Appendix D). This is a sanity check on model-fitting correctness, not a claim about real-world accuracy.

Does the “Intervention Suggestion” stage give medical advice? No — it is a transparent, rule-based heuristic identifying the input feature with the largest standardized deviation from a population baseline (§4.6). It is explicitly not a causal-inference or clinical recommendation system.

What license governs reuse of this paper and the codebase? Both are MIT-licensed (`LICENSE`). See Appendix F for the full governance and contribution model.

How do I cite VitaGraph? See the Citation section of `README.md` and the machine-readable `CITATION.cff` ; a DOI will be added once the repository is archived on Zenodo (`.zenodo.json`).

Appendix F — Governance, Licensing, and Contribution Model

License. VitaGraph (code, documentation, and this white paper) is released under the MIT License (`LICENSE`). This permits reuse, modification, and redistribution, including commercial use, with attribution and without warranty.

Maintainership. The project is currently maintained by its original author, [@Ciprian-LocalPulse](#). Community contributions are welcomed under the process in `CONTRIBUTING.md` and the standards in `CODE_OF_CONDUCT.md` (Contributor Covenant v2.1).

Versioning. The project follows [Semantic Versioning](#); the public API surface for versioning purposes is the four classes re-exported from `vitagraph/__init__.py` plus the documented `vitagraph` CLI flags (§3.3). Releases are cut via signed Git tags (`vX.Y.Z`), which trigger automated PyPI publication and GitHub Release creation (§5).

Security disclosure. Vulnerabilities should be reported privately per `SECURITY.md` (GitHub Security Advisories or direct maintainer contact), not as public issues.

Long-term intent. Per §7, the maintainer’s stated intent is to keep VitaGraph’s synthetic-data core stable and well-documented as a teaching/reference artifact, while treating any real-data extension as a separate, ethically- and legally-gated effort (`docs/ROADMAP.md` , `data/README.md`) rather than an incremental feature added without that groundwork.

VitaGraph is released under the MIT License. See `LICENSE` , `CITATION.cff` , and `SECURITY.md` in the repository root for licensing, citation, and vulnerability-disclosure details. Corrections and clarifications to this document are welcome via the process described in `CONTRIBUTING.md` .